Arduino IDE 双节点工程与烧录手册（V1.1）

两套工程均采用Arduino框架，并整理成Arduino IDE可直接打开的工程。不需要PlatformIO。已用Arduino CLI（Arduino IDE 2底层同类构建工具）编译；未连接用户实物烧录。

1. 安装前准备

Windows10电脑，Arduino IDE 2.x，支持数据传输的Micro-USB线，两块ESP开发板。ESP32图片中为CP2102，ESP8266图片中疑似CH340系列，以实际芯片为准。只从芯片厂商或开发板供应商可信来源安装对应驱动。Windows设备管理器中应出现可用COM端口。

编程时把ESP从基板上取下，单独USB连接电脑。烧录完成先拔USB，再断电插回基板，最后接外部5V。基板草案不支持USB与外部5V同时接入，不能假定任意NodeMCU板都有电源防反灌。

2. 解压并设置工程位置

解压工程后有以下目录，保留完整层级：

```
arduino/
Room101_ESP32/
Room101_ESP32.ino
NodeRuntime.cpp
alarm.h
secrets.h
Room102_ESP8266/
Room102_ESP8266.ino
NodeRuntime.cpp
alarm.h
secrets.h
libraries/
AsyncMqttClient/
AsyncTCP/
ESPAsyncTCP/
ArduinoJson/
DHT_sensor_library/
Adafruit_Unified_Sensor/
```

在Arduino IDE“文件→首选项”中，将“项目文件夹位置／Sketchbook location”设置为解压后的`arduino`文件夹，重启IDE。这样libraries中的固定版本库会被发现。不要只复制.ino文件而丢失.cpp和.h文件。

如需保留原项目文件夹，也可将本包libraries内六个目录复制到原Sketchbook的libraries中。已有同名但版本不同的库先备份并移出libraries，避免重复库选择。不要把整个libraries文件夹当一个ZIP库导入。

.ino为工程入口和板型校验，完整setup()/loop()在同窗口的NodeRuntime.cpp标签中。这是Arduino支持的多文件工程，不是空程序。

3. 安装开发板包

“文件→首选项→附加开发板管理器网址”填入两行：

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

```
https://arduino.esp8266.com/stable/package_esp8266com_index.json
```

打开“ 工具→开发板→开发板管理器 ”，分别安装：

工程	开发板包	本次验证版本	选择板型
Room101_ESP32	esp32 by Espressif Systems	2.0.17	ESP32 Dev Module
Room102_ESP8266	esp8266 by ESP8266 Community	3.1.2	NodeMCU 1.0 (ESP-12E Module)

这里的ESP32是经典ESP32-D0WD-V3，不选ESP32-C3/S3。先按验证版本使用，不直接升级ESP32 3.x及异步网络库后假定兼容。两块实际Flash容量未实测，参考4MB需核验。

随包库版本：AsyncMqttClient0.9.0、AsyncTCP1.1.1（ESP32）、ESPAsyncTCP1.2.2（ESP8266）、ArduinoJson6.21.5、DHT sensor library1.4.6、Adafruit Unified Sensor1.1.15。没有使用PubSubClient，不要拿另一个库替换。

4. 修改各自secrets.h

在IDE打开对应.ino，切到secrets.h，填写：

- WIFI_SSID：比赛路由器2.4GHz无线名称。

- WIFI_PASSWORD：无线密码。

- MQTT_HOST：树莓派在比赛局域网中的实际IPv4地址，不带http://及端口。

- DEVICE_ID：101为room101-esp32，102为room102-esp8266，保持不变。

- MQTT_PASSWORD：使用树莓派安装生成的对应节点密码，不是网页管理员密码。

示例中的CHANGE_ME必须替换，真实密码不要投屏或公开。树莓派管理员可用`sudo cat /etc/roomguard/nodes.env`在私密环境中查看节点密码。

HARDWARE_PROFILE=0表示原型接线（ESP32 AO10k/15k），=1表示V1.1基板参考（ESP32 AO20k/10k）。只有实物按对应电路完成并核验后才改。ESP8266的A0量程和56k/10k分压仍需分别核验。不要仅改软件比例而保持错误的硬件接线。

MQ_WARMUP_MS默认180000只是开发预热值，MQ-2首次老化和每次开机稳定时间以实物资料与实测为准。温湿度不触发告警。

固件固定引脚表（GPIO编号）

模块信号	ESP32／101	ESP8266／102
DHT11 DATA	GPIO27	GPIO2／D4，注意启动上拉
MQ-2 AO	GPIO34／ADC1	A0，先核验量程
MQ-2 DO	GPIO26	GPIO14／D5
火焰DO	GPIO25	GPIO12／D6
蜂鸣器控制	GPIO23	GPIO13／D7
RGB红R	GPIO18	GPIO4／D2

模块信号	ESP32／101	ESP8266／102
RGB绿G	GPIO19	GPIO5／D1
RGB蓝B	GPIO21	GPIO16／D0

本表是ESP逻辑侧连接，5V模块信号必须经过相应适配，不能按表把MQ输出直接接GPIO。原型与基板的分压比例分别核验。

5. ESP32编译和上传

1. 打开Room101_ESP32.ino。

2. 选择ESP32 Dev Module，选择插入此板后出现的COM端口。

3. 建议Upload Speed115200，CPU240MHz，Flash4MB（待实测），Flash ModeDIO，默认分区方案；其他先用默认值。

4. 点击“ 验证 ”（对勾），确认编译成功，再点“ 上传 ”（右箭头）。

5. 若一直Connecting，可按住BOOT开始上传，看到开始写入后松开；必要时按一次EN复位。不能把外部5V接在板上同时操作USB。

6. 打开串口监视器，选择115200。固件当前不逐条打印所有传感器数据；以启动信息、服务器在线状态及实际声光核验为主。

6. ESP8266编译和上传

1. 打开Room102_ESP8266.ino。

2. 选择NodeMCU1.0（ESP-12E Module）和实际COM端口。

3. Upload Speed115200，CPU80MHz；Flash Size参考4MB（FS2MB或IDE默认LittleFS配置，以实物容量核验）。配置需要LittleFS持久保存解除时间。

4. 验证、上传、观察成功提示。一般自动进入烧录；必要时按住FLASH、短按RST后重试，再松开FLASH。

5. ESP8266的D4/GPIO2连DHT，必须保持启动上拉，避免外设低电平导致不启动。首次调试可先不插传感器逐项验证。

切换节点时同时切换工程、板型和端口，避免上一块板的设置沿用到另一块。两份固件不能互相刷入。

7. 烧录后的最小验证

拔USB，断电插回已经检查过的基板或原型电路。选择正确电源与VREF跳帽，只装一块ESP，外部5V上电。

检查：有正常供电→等待预热→网页出现对应房间在线→温湿度有效→受控烟雾／火焰输入触发对应声光→远程消音有回执→断网但保持供电仍报警→恢复网络重新上报。记录每一步，不能只因上传成功就判定系统完成。

8. 常见错误

现象	排查
没有COM端口	数据线、USB口、驱动、设备管理器；某些线只能充电
找不到AsyncMqttClient.h等	Sketchbook位置、libraries目录层级、是否重启IDE、重复版本
StaticJsonDocument报错	核对ArduinoJson6.21.5，未切到其他大版本
AsyncTCP报错	核对ESP32核心2.0.17和AsyncTCP1.1.1；ESP8266使用ESPAsyncTCP
上传占用／拒绝访问	关闭其他IDE、串口监视器和占用COM的软件
上传后一直离线	Wi-Fi2.4GHz、实际服务器IP、节点ID与对应MQTT密码、代理ACL和供电
ESP8266不上电运行	GPIO2上拉、BOOT脚、电源、复位、模块插向
Linux构建缺serial模块	在构建使用的Python环境安装pyserial；Windows官方板包通常带专用工具

本包Arduino CLI编译日志在artifacts/v1.1，属于构建验证；实际USB烧录与整机验收需要在用户设备上完成。